ORIGINAL ARTICLE

Keyframe-Anchored Object Maps: Registering Late Zero-Shot 6D Pose Estimates in Asynchronous SLAM

Abstract

Zero-shot 6D pose estimators recognize objects from CAD models alone but take up to seconds per frame. Running them on a second computer keeps SLAM real-time, yet results arrive 0.7–6.6 s late, from another camera and clock, after loop closure may have corrected the map. This study presents a two-computer system that turns such late results into a persistent object map. Its core, capture-time keyframe anchoring, streams the reference keyframe of every SLAM pose with re-sent keyframe lists and stores each object relative to the keyframe current at image capture. Metadata timestamps with joint offset–extrinsic calibration, a text-prompted and gate-verified SAM-6D, an existence-probability state manager, and a latency-compensated display complete the system. On 61 replayed live runs, anchoring reduces the median deviation of repeated observations from 9.67 to 1.51 cm and, when the map is corrected in flight, the share above 10 cm from 30.8% to 1.5%. Indoors, 72 of 74 reappearing objects are re-associated (34 without the map).

KEYWORDS

asynchronous sensor fusion, loop closure, object-level mapping, simultaneous localization and mapping, zero-shot 6D object pose estimation

1 | INTRODUCTION

Service and logistics robots, digital twins, and XR applications need *object-level* maps: the objects in a space, each with a 3D position and orientation in the map frame, kept up to date while the platform moves. Two mature building blocks exist. Visual and LiDAR SLAM systems such as ORB-SLAM3 [1] and HDL-Graph-SLAM [2] estimate the sensor pose in real time and remove accumulated drift by loop closure and graph optimization. Zero-shot 6D object pose estimators such as SAM-6D [3], FoundPose [4], and GigaPose [5] estimate the pose of an object unseen in training, given only its CAD model.

Combining the two is not a matter of multiplying two transforms. Zero-shot estimators are slow; SAM-6D segments the whole image into hundreds of proposals and needs about 0.9 s per frame in our setting. When the estimator shares a computer with SLAM, SLAM loses its time budget: in our system, the delivery latency of SLAM poses rose from 0.04 s to 2.5 s when a heavy process ran on the same computer. A slower SLAM tracks less reliably and triggers larger map corrections, which move every object registered on the map. Fusion++ [6], which runs recognition and SLAM together, operates at 4–8 Hz overall.

Moving recognition to a second computer keeps SLAM real-time but creates five problems at once: (i) the two computers have different clocks; (ii) the two cameras have different poses; (iii) a recognition result arrives hundreds of milliseconds to seconds after its image is captured, so registering it with the camera pose at *arrival* displaces the object by the platform motion in between; (iv) while the result is in flight, SLAM may correct past keyframe poses by loop closure or cull keyframes; and (v) under network load, the SLAM pose for the capture time may itself arrive *after* the recognition result.

Problem (iv) is the least obvious. Interpolating the trajectory at the capture timestamp, as a timestamped transform buffer such as ROS tf [7] does, solves (iii) but stores the object in the map frame; once loop closure moves the trajectory, the stored object stays behind. Figure 1 (blue panel) shows a real case: after a loop closure that moved keyframes by up to 3 m, 68 of 119 results stored in this way lie more than 10 cm from their final object position, versus 6 of 119 with the anchoring proposed here. Anchoring content to keyframes is known in augmented reality, and ORB-SLAM3 moves map points with their reference keyframes, but in both cases the anchored entity is created from the current frame. For a result computed seconds after its image, the question of *which keyframe*, *at which time*, has not been addressed. The contributions of this study are as follows:

- **Capture-time keyframe anchoring.** The SLAM computer streams, with every pose, the reference keyframe used to track that frame and re-sends the full keyframe list periodically, right after each map change, and 0.5 s and 3 s later. The recognition computer stores each late result relative to the reference keyframe at capture time, waits when the matching pose has not arrived, and re-anchors objects of culled keyframes.
- **Cross-device time and extrinsic alignment** using per-frame camera metadata timestamps, a round-trip clock offset, and a three-stage calibration that estimates the residual time offset jointly with the camera–camera extrinsic.
- **A real-time, verifiable zero-shot recognizer**: SAM-6D with text-prompted proposals, sequential rejection gates, and pose verification that can report that an object is absent.
- **A persistent keyframe-anchored object map and a latency-compensated display**, evaluated on YCB-Video [8], YCB-M [9], and indoor runs with two hardware-synchronized RGB-D cameras and a LiDAR, including the limitations of the system.

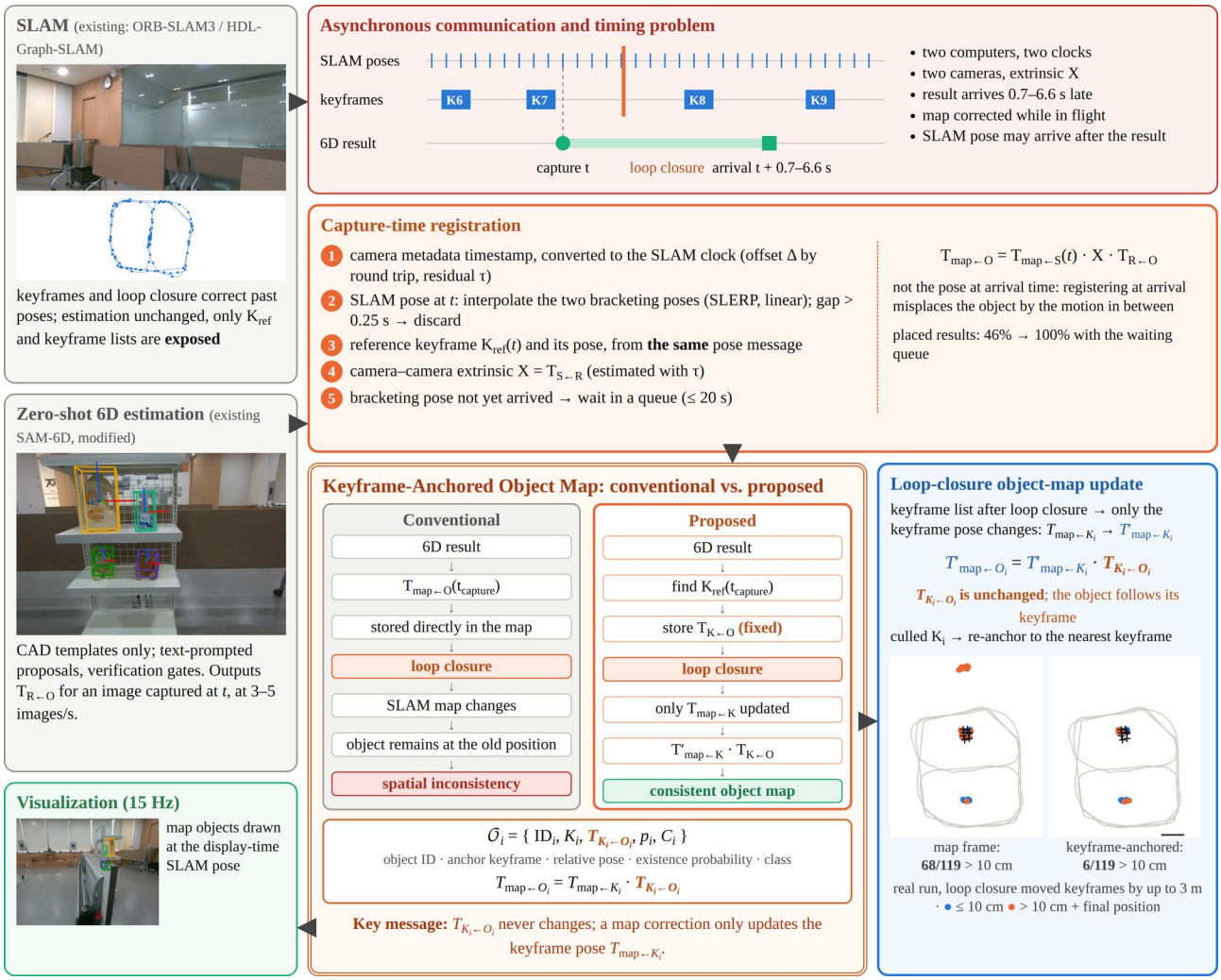


FIGURE 1 Overview, with panel areas in proportion to their role. Existing SLAM and zero-shot 6D estimation (gray) run on two computers, so results arrive late, from another clock and camera, while the map may be corrected (red). Each result is registered at its capture time (orange) and stored relative to the reference keyframe of that time (double frame): unlike conventional map-frame storage, the object then follows the keyframe correction after loop closure (blue). The maps in the blue panel are from a real run whose loop closure moved keyframes by up to 3 m. All images are from the recordings and logs.

2 | RELATED WORK

Zero-shot 6D pose estimation. CNOS [10] and SAM-6D [3] segment the image with SAM [11] or FastSAM [12], describe each proposal with DINOv2 [13] features, and match it against templates rendered from the CAD model; SAM-6D then estimates the pose by two-stage point matching. FoundPose [4], GigaPose [5], and FoundationPose [14] follow related template- or model-based designs. These methods output camera-frame poses for single images and do not maintain objects over time, and their proposal stage dominates the run time. Open-vocabulary detectors such as YOLO-World [15] produce class-level boxes from text prompts in tens of milliseconds; we use one as the proposal generator in front of instance-level verification.

Object SLAM and object maps. SLAM++ [16], CubeSLAM [17], and QuadricSLAM [18] use objects as landmarks to improve localization, relying on pre-trained detectors or category-level shapes. CosyPose [19] aggregates multi-view object poses into a consistent scene. Fusion++ [6], POCD [20], and Khronos [21] maintain object-level maps with existence or change models. Our work is complementary: objects are not fed back into SLAM; SLAM estimation is left unchanged, and CAD-defined objects follow its corrections.

Asynchronous perception. MaskFusion [22] runs instance segmentation asynchronously on a separate GPU of the same computer, assuming one camera and one clock. Edge-assisted mobile AR [23] offloads detection to a server and compensates the returned boxes by image tracking, without a map. Transform libraries [7] interpolate poses at a requested time but do not revisit stored results after the trajectory is optimized. To the best of our knowledge, no prior system binds late, off-device recognition results to the SLAM keyframe of their capture time so that the object map absorbs corrections made while the result is in flight.

3 | SYSTEM OVERVIEW

Figure 1 shows the system. Device 1 runs SLAM with its own sensor: an RGB-D camera for ORB-SLAM3 or a LiDAR for HDL-Graph-SLAM. Device 2 has a GPU and a second RGB-D camera for recognition. The computers are connected by a wired LAN, and the two cameras on

the cart are hardware-synchronized (Fig. 2). Notation: $T_{A \leftarrow B}$ maps coordinates from frame B to frame A; X = $T_{S \leftarrow R}$ is the extrinsic from the recognition camera R to the SLAM camera S; for an image captured at time t, the estimator outputs $T_{R \leftarrow O}$ for object O.

4 | METHOD

4.1 | Time and extrinsic alignment



FIGURE 2 Data-collection cart with two RGB-D cameras (640 × 480, 30 Hz) connected by a sync cable and a 16-channel LiDAR (10 Hz).

Capture timestamps. We use the timestamp each camera attaches to every frame's metadata (for the RGB-D cameras used, the hardware clock mapped to the host clock, or the host arrival time), read on receipt in live operation and from the recording during replay. Recorder association stamps are unusable: because of frame drops (100–200 ms gaps), they deviated from the true frame times by −2.1 s to +3.8 s, whereas metadata timestamps agree with the frame times within 1.4 ms (SLAM camera) and 0.1 ms (recognition camera).

Clock offset. Device 2 sends a UDP packet at t_0, Device 1 replies with its clock reading t_r, and Device 2 receives the reply at t_1. The offset

$$\Delta = t_r - (t_0 + (t_1 - t_0) / 2) \qquad (1)$$

is taken as the median over repetitions. The error of the symmetric-delay assumption is bounded by half the round-trip time; with a measured round trip of 3.08 ms, it is at most 1.54 ms, well below the 20 ms color–depth pairing tolerance. All timestamps are expressed on the Device 1 clock.

Residual offset and extrinsic. A residual offset τ remains because of different time bases and transport delay. We estimate τ together with X from a calibration drive in which each camera runs its own visual odometry, in three stages chosen by observability under planar motion: (1) the normal of the plane fitted to each trajectory gives gravity (out-of-plane RMS 0.60 cm and 0.20 cm); (2) relative motions over 0.5–4 s windows form AX = XB constraints, from which planar translation, yaw, and τ are found by a Huber-weighted fit while τ is scanned (e.g., ±60 ms in 2 ms steps; 14 592 pairs, median residual 0.41 cm); (3) the height difference, unobservable from planar motion, is measured from the floor plane in each depth image (1.107 m and 0.998 m). τ was 0 to +4 ms in three calibration drives, −48 ms in another recording, and 0.196 s in a recording with a mis-set recorder clock; all were recovered by the same search. The same procedure gives the LiDAR–camera extrinsic (3122 pairs, residual 1.18 cm and 0.11°).

4.2 | SLAM side: real-time tracking and keyframe streaming

Real-time budget. With 2000 ORB features per frame, the mean tracking time of ORB-SLAM3 was 24.7 ms, and 170 of 7142 frames exceeded the 33.3 ms camera period. Profiling showed that the bottleneck is overhead proportional to the number of map points rather than feature extraction or optimization iterations. Seven result-preserving changes (removing per-frame copies for a disabled viewer, in-place aggregation of map-point observations, allocation-free grid search, POPCNT-based descriptor distance, one lock instead of four per map-point query, a smaller distribution-tree reservation, and parallel extraction per pyramid level) reduce the mean to 15.4 ms and over-period frames to zero, while the loop-return error of a figure-eight drive stays at 26.6 cm versus 26.5 cm (Fig. 3). When processing falls behind, the newest frame is processed and the number of skipped frames is reported.

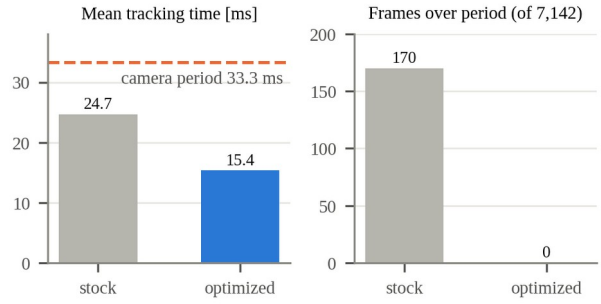


FIGURE 3 ORB-SLAM3 with 2000 features before and after the result-preserving optimizations.

Pose message. For every frame, Device 1 sends the capture timestamp, tracking state, $T_{map \leftarrow S}$ (or *no pose* when tracking failed or, in prior-map mode, when the pose is not anchored to the map), the identifier and pose $T_{map \leftarrow K}$ of the reference keyframe used to track the frame, the map identifier, a map-change flag, and the number of skipped frames. The pose and reference keyframe are read in the tracking thread immediately after tracking, so they refer to the same instant. Because ORB-SLAM3 does not expose this information, we added read-only accessors. The median pose delivery latency is 18.1 ms.

Keyframe lists. Loop closure in ORB-SLAM3 is followed by a global bundle adjustment that finishes seconds later in another thread, so a single list sent right after the loop misses the final correction. Device 1 therefore sends the list of all live keyframes (identifier and 3 × 4 pose, about 14 kB per 100 keyframes) every 1 s, right after a frame flagged with a map change, and

again 0.5 s and 3 s later, merging coincident sends. A keyframe missing from the list is treated as culled, so no separate culling message is needed. For HDL-Graph-SLAM, we added a publisher of all graph-optimized keyframe poses, so the same anchoring applies to LiDAR SLAM.

4.3 | Text-prompted, gate-verified 6D pose estimation

Figure 4 compares the recognizer with SAM-6D. Templates are rendered from 42 viewpoints. In addition to the DINOv2 class token and pose features of SAM-6D, we precompute mid-layer DINOv2 patch features, a 16 × 8 hue–saturation histogram with a render-to-camera color correction applied on the template side, and one text embedding per object. Adding an object requires only its CAD model and one precomputation pass.

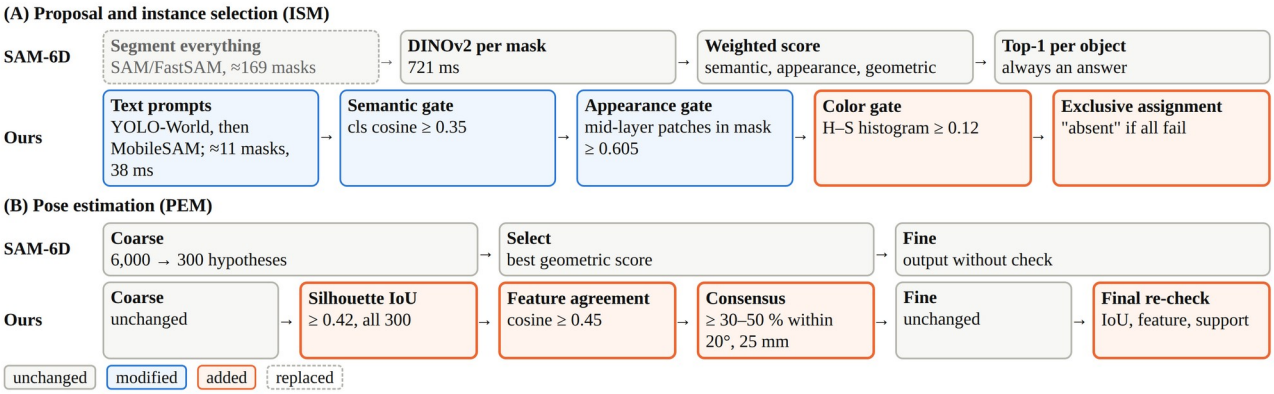


FIGURE 4 Recognizer compared with SAM-6D. (A) Text-prompted proposals and sequential gates replace segment-everything proposals and weighted-sum scoring. (B) Pose verification is inserted before and after refinement.

Proposals. All object prompts (e.g., "milk carton", "yellow can") are given to YOLO-World [15] in a single pass; boxes above a low confidence (0.02) are kept, and a box matching several prompts is kept for each. MobileSAM [24] segments the object inside each box. Proposals per frame decrease from 169 to 11 and proposal feature extraction from 721 ms to 38 ms; on 193 frames without target objects, frames with false detections decrease from 120 to 2 (recall on 118 frames with targets, 0.949 versus 0.915).

Sequential gates. Instead of a weighted sum followed by the top-1 proposal per object, each proposal must pass, in order: (1) a semantic gate, the mean cosine of the five best template class tokens ≥ 0.35; (2) an appearance gate, the mean best-match cosine of mid-layer (blocks 2 and 9) patch features inside the mask ≥ 0.605; (3) a color gate, the Bhattacharyya similarity of hue–saturation histograms (value channel excluded) ≥ 0.12; and (4) exclusive assignment, by which one box is assigned to one object only, with the color score deciding between objects whose templates are mutually similar. An object without a surviving proposal is reported absent.

Pose verification. The coarse (6000 → 300 hypotheses) and fine stages of the SAM-6D pose estimation model are unchanged. Before refinement, all 300 hypotheses are checked by silhouette IoU between the projected CAD points and the mask (≥ 0.42), the mean feature cosine between observed and corresponding CAD points (≥ 0.45), and consensus (at least 30%–50% of survivors within 20° and 25 mm of one pose). The refined pose is checked again.

4.4 | Keyframe-anchored object map

Capture-time registration. For a result whose image was captured at t,

$$T_{map\leftarrow O} = T_{map\leftarrow S}(t) \cdot X \cdot T_{R\leftarrow O} \quad (2)$$

where $T_{map\leftarrow S}(t)$ interpolates the two SLAM poses bracketing t (spherical linear interpolation for rotation, linear for translation); detections whose bracketing poses are more than 0.25 s apart are discarded. The object is stored relative to the reference keyframe K of the bracketing pose nearest to t, using the keyframe pose sent *with that pose message*,

$$T_{K\leftarrow O} = T_{map\leftarrow K}(t)^{-1} \cdot T_{map\leftarrow O} \quad (3)$$

and is always used through the latest keyframe pose T′ received in the keyframe lists:

$$T'_{map\leftarrow O} = T'_{map\leftarrow K} \cdot T_{K\leftarrow O} \quad (4)$$

Each object is thus stored as $O_i = \{ID_i, K_i, T_{K_\{i}\leftarrow O_i\}, p_i, C_i\}$, with identifier ID_i, anchor keyframe K_i, relative pose $T_{K_\{i}\leftarrow O_i\}$, existence probability p_i, and class C_i. A map correction changes only $T_{map\leftarrow K_\{i}\}$; $T_{K_\{i}\leftarrow O_i\}$ is never modified.

Because $T_{map\leftarrow S}(t)$ and $T_{map\leftarrow K}(t)$ come from the same tracking state, (3) is invariant to any correction applied to K after t, including corrections made while the result is in flight (Fig. 1). Anchoring to the keyframe current at *arrival* instead pairs a pre-correction camera pose with a post-correction keyframe pose. If the bracketing SLAM poses have not arrived, the result waits in a queue for up to 20 s. If K is culled, the object is re-anchored to the nearest live keyframe. The map can be rebuilt from all recorded observations whenever keyframe poses change (75 ms for 308 frames).

Association and update. A detection is associated with an existing object of the same class within 0.15 m (three times wider for tentative objects), nearest first and one-to-one per frame. Rotation is excluded from association because zero-shot estimators often return poses flipped by 180° for box-like objects, which would

split one object into two. Position is the running mean of observations; orientation is the mode of observations clustered within 30°, after aligning rotationally symmetric objects to the equivalent rotation nearest to the reference.

Existence probability. Each object has an existence probability p and a state: tentative (starting at p = 0.7, hidden), active (at least two observations and p > 0.6), lost (p < 0.3), remembered (p < 0.05 with at least five observations), or deleted. A miss is counted only if the object projects inside the view *and* lies within the detection range of the recognizer (e.g., 1.5–2.2 m for small objects); p is then updated by Bayes' rule with a detection probability of 0.08–0.5 and a false-alarm rate of 0.01–0.1. An object is deleted as removed when the depth image shows free space at least 15 cm behind it over 95% of a 15 cm window in three consecutive frames. Lost and remembered objects remain association candidates, so a returning object regains its identifier. When two objects of one class exist and one has at least five times the observations of the other, the smaller one is hidden as a duplicate; this removes high-confidence misreadings (scores 0.96–1.00) that score thresholds cannot.

4.5 | Latency-compensated display

Device 2 keeps the last 120 recognition-camera images with their capture timestamps. At 15 Hz, it sets the display time to the newest image time minus a display delay, selects the buffered image at that time, interpolates the SLAM pose at that time, and projects all objects in the map into that image. The image and boxes are therefore consistent; only the display as a whole is delayed. The delay follows the 90th percentile of recent SLAM-pose arrival delay plus 0.15 s, bounded to 0.3–3 s, increased immediately and decreased by 2% per step. The delay applies to the display only; the map used by a robot is updated with the newest results and keyframe poses.

5 | EXPERIMENTAL SETUP

Platform. The cart (Fig. 2) carries two RGB-D cameras and a 16-channel LiDAR. Recorded sessions are replayed in real time (1×) on two computers connected by a wired LAN, each replaying its own sensor stream from a common start time. Device 1 is a laptop with an Apple M4 Pro processor running ORB-SLAM3 natively; Device 2 is a laptop with an Intel Core i9-12900HK and an NVIDIA GeForce RTX 3080 Ti Laptop GPU [verify per run]. Single-image comparisons (Table 1) use an NVIDIA GeForce RTX 5090 [verify]. In all runs, SLAM builds its map from scratch.

Data. *YCB-Video* [8]: 12 test videos; 900 images of the BOP [25] test list for single-image evaluation and all 20 738 frames played at 30 fps for live evaluation. *YCB-M* [9]: 7252 images of 32 static scenes recorded by a robot-arm camera; 925 images (every eighth) for single-image evaluation; playback speed is set from the inter-frame camera motion (about 13 mm) because timestamps are not provided. *Indoor runs*: eight objects (milk carton, snack box, spray bottle, mug, jug, can, teddy bear, dinosaur doll) in indoor spaces: a dark figure-eight run (284 s), a 91 s run (1883 frames, 96 map corrections), a bright-hall figure-eight run (151 s), and a large figure-eight run (315 s). Indoor runs have no ground-truth object poses; we use the consistency of repeated observations and pseudo ground truth built from a separately optimized SLAM trajectory of the recognition camera.

Metrics. A pose is correct if ADD-S is below 10% of the object diameter. *Found* is the share of visible ground-truth objects with a correct answer; *correct answers* is the share of answers that are correct; BOP AR is computed with the official toolkit. The *on-screen match rate* is the share of visible ground-truth objects whose displayed box is correct in each frame. *Deviation* is the distance of an observation, registered in the final map, from the median position of its object.

6 | RESULTS

6.1 | Recognition

Table 1 compares SAM-6D (official code, FastSAM proposals) with our recognizer using text prompts and with ground-truth boxes substituted for the text detector, which isolates the effect of proposal recall. Our recognizer is correct in 94%–100% of its answers on all three datasets, produces at most one seventh of the wrong answers per image, and is 3.7–5.6 times faster. On the indoor runs, it also attains a higher BOP AR (37.9 versus 33.3); on YCB, the text detector misses objects, so recall and AR are lower than those of SAM-6D.

TABLE 1 Single-image comparison without SLAM (one answer per object per image).

Dataset / method	Found (%)	Correct answers (%)	Wrong per image	ADD-S AUC	BOP AR	Time (ms)
YCB-V · SAM-6D	90.2	90.9	0.41	90.0	78.7	1613
YCB-V · ours, text	63.2	98.6	0.04	62.0	57.8	433
YCB-V · ours, GT box	80.9	100.0	0.00	79.0	74.5	528
YCB-M · SAM-6D	79.1	80.0	1.01	81.4	52.3	2197
YCB-M · ours, text	42.6	94.0	0.14	41.9	29.7	395
YCB-M · ours, GT box	65.8	95.9	0.14	64.2	46.6	519
Indoor · SAM-6D	52.4	43.0	3.39	53.6	33.3	2884
Indoor · ours, text	51.8	94.7	0.14	50.5	37.9	557
Indoor · ours, GT box	52.2	97.6	0.06	49.4	37.2	497

Abbreviations: GT, ground truth; AUC, area under the curve (0–10 cm); AR, average recall. YCB-V, 900 images; YCB-M, 925 images; indoor, 917 images with pseudo ground truth. Time is the median per image.

Figure 5 isolates the gates on 959 human-labeled (frame, object) cells from an indoor run. The color gate removes most false positives (310 → 63); the appearance gate raises F1 but adds false positives (96), which exclusive assignment removes (28). With per-object hue correction, F1 reaches 0.841. Pose verification accepts 288 and rejects 103 of 391 results in one run; each rejected pose would have been output by top-1 selection. End-to-end time from proposal to verified pose decreases from 1445 ms to 377 ms under real conditions.

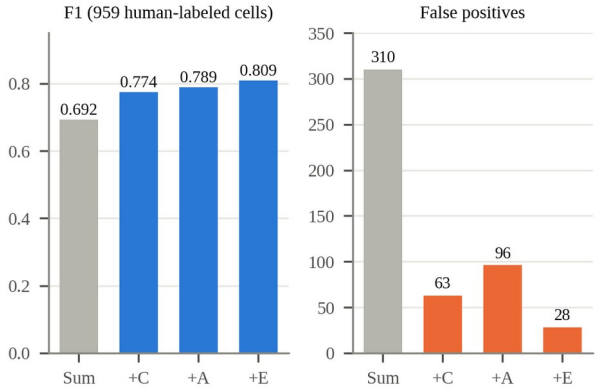


FIGURE 5 Effect of adding gates one at a time on 959 human-labeled cells: weighted sum (Sum), then color (+C), appearance (+A), and exclusive assignment (+E).

6.2 | Capture-time keyframe anchoring

We replay 61 recorded live ORB-SLAM3 runs (three indoor recordings; 8964 results; capture-to-arrival latency median 1.45 s, maximum 6.6 s) with SLAM poses, keyframe updates, and results in their actual arrival order, and register the same results in four ways: (a) camera pose at arrival; (b) capture-time interpolated pose stored in the map frame, as a transform buffer would; (c) as in (b) but anchored to the keyframe current at arrival; and (d) anchored to the capture-time reference keyframe (proposed). Deviation is measured in the final map after all corrections (Table 2, Fig. 6).

TABLE 2 Deviation of repeated observations in the final map.

Registration	Median (cm)	90th pct. (cm)	> 10 cm (%)	90th pct., corrected (cm)	> 10 cm, corrected (%)
(a) Arrival-time pose	9.67	31.94	48.2	294.0	63.8
(b) Capture time, map frame	3.08	9.70	9.2	299.8	30.8
(c) Capture time, arrival-time keyframe	1.59	5.36	3.6	7.80	5.4
(d) Capture-time keyframe (proposed)	**1.51**	**5.21**	**3.5**	**5.81**	**1.5**

Corrected: the 130 results whose wait spanned a map correction larger than 2 cm. All: 8964 results.

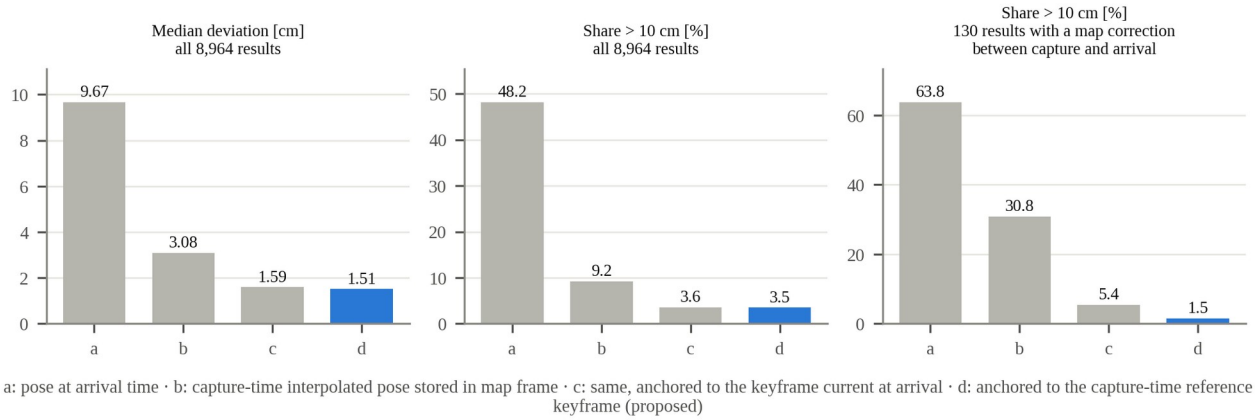


FIGURE 6 Registration strategies (a)–(d) on 61 replayed live runs.

Using the capture time removes most of the error (a → b). Without corrections in flight, (c) and (d) are similar, but among the 130 results whose wait spanned a map correction, only (d) remains consistent; for the 18 runs of the dark figure-eight, the median for these cases is 5.02 cm for (c) and 0.58 cm for (d). Across a loop closure in that run, the position difference of the same object before and after the loop decreases from 5.4 cm (map-frame storage) to 0.8 cm. In the 91 s run, 96 map corrections of up to 35 cm occur over 92 keyframes; with anchoring

disabled, three objects leave copies 30–32 cm away, giving 11 locations for eight objects. The waiting queue raises the share of results placed on the map from 46% to 100%.

6.3 | Persistent object memory

In two indoor runs, 74 events occur in which an object leaves the view or is occluded and reappears (39 out of view). Of these, 72 are re-associated with the same map object, whereas association in camera coordinates succeeds for 34 (Table 3). The two failures occur before a loop closure, when drift displaces the object by 31–33 cm, beyond the association radius. On YCB-M (101 reappearances after at least 1 s), map-based association succeeds in 91.8%–97.9% of eligible cases, 3–7 percentage points above camera-frame association.

TABLE 3 Re-association in indoor runs (object map / without map).

	Bright hall, 151 s	**91 s run**
Reappearances (out of view)	35 (18)	39 (21)
Re-associated	35 / 22	37 / 12
Identifiers created (8 objects)	8 / 51	10 / 68
Same-class duplicates after run	0 / 17	0 / 19

Majority-vote orientation reduces the share of displays off by more than 20° or 50 mm from 23.9% (mean orientation) to 9.3%. Duplicate suppression hides misreadings about 1.8 m from the true object at observation ratios of 65:3, 72:3, and 68:4 without changing the display of correct objects. Range-aware miss counting raises the display ratio of a small mug and can in the large figure-eight run with HDL-Graph-SLAM from 37%/46% to 83%/82%. In live YCB-Video playback with the same SLAM and map, SAM-6D results produce eight duplicates over 12 scenes and a maximum position error of 85.1 mm, whereas our recognizer produces none (maximum 23.8 mm with ground-truth boxes); wrongly drawn boxes decrease from 0.266 to 0.001 per frame.

6.4 | Latency-compensated display

Figure 7 compares holding the last result in camera coordinates with drawing the object map at the display-time SLAM pose. The match rate increases for every recognizer and dataset, and the effect grows with recognizer latency: on YCB-M, where SAM-6D (2.0 s per image) processes only 8% of frames, it rises from 3.2% to 72.7%. With the adaptive delay, the share of display frames without a SLAM pose to interpolate decreases from 100% to 0% under high network delay. In the 91 s run, recognition runs at 3.2 images/s with a median latency of 0.70 s, while the display is updated at 15 Hz and SLAM tracks all 1883 input frames.

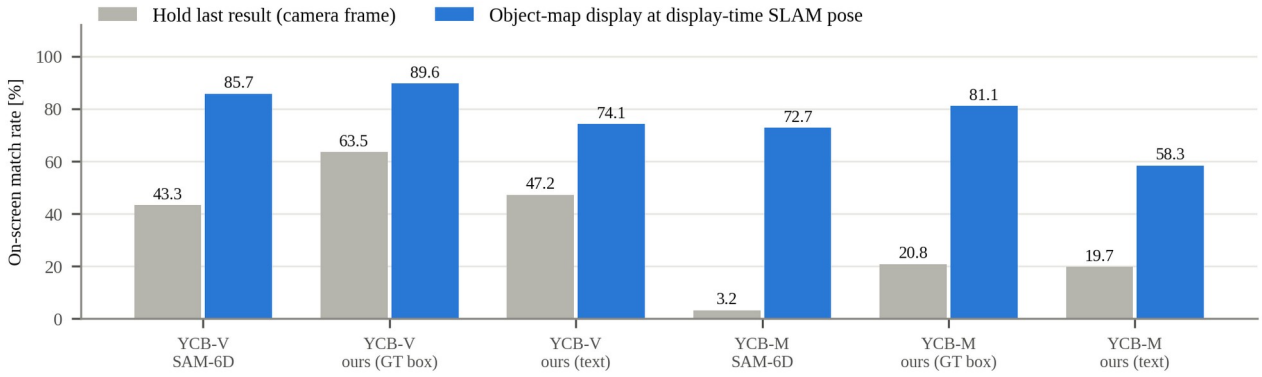


FIGURE 7 On-screen match rate on YCB-Video (YCB-V) and YCB-M.

6.5 | Limitations

For objects 30%–50% visible, SAM-6D finds 46% on YCB-Video but our recognizer only 3%, because occluders inside the box lower appearance scores and small silhouettes fail the overlap threshold; for well-visible objects in the indoor runs, the rates are 75% (ours) and 70%. Text prompts miss some objects, and texture-less objects fail the gates even with ground-truth boxes. Anchoring follows SLAM corrections, including wrong ones: in 40 runs of the large figure-eight with identical input, two loop closures damaged an accurate map (median deviation 17.0 and 54.4 cm against a LiDAR scan-matching reference; the latter increased displayed objects from eight to 13), whereas 34 runs agreed within 2.1–2.6 cm. Scenes with several objects of one class were not evaluated.

7 | DISCUSSION

The results separate three effects. First, *which time* is used: replacing arrival time with capture time accounts for most of the improvement (9.67 → 3.08 cm). Second, *which frame* stores the object: keyframe-relative storage is needed when SLAM corrects its past. Third, *which keyframe*: the arrival-time keyframe is adequate most of the time but fails when a correction occurs in flight, because it pairs a pre-correction camera pose with a post-correction keyframe pose (30.8% → 5.4% → 1.5% above 10 cm). Because zero-shot recognizers are slow and their latency varies, this case is not rare; it occurred 130 times in our runs. The SLAM side only exposes information it already maintains and schedules its transmission, so the approach applies to other keyframe- or graph-based SLAM systems, as shown with ORB-SLAM3 and HDL-

Graph-SLAM, and to other recognizers that output camera-frame poses.

8 | CONCLUSION

This study presents a two-computer system that keeps SLAM real-time and places late zero-shot 6D pose estimates correctly in the SLAM map. Capture-time keyframe anchoring, supported by keyframe streaming, metadata timestamps with joint offset and extrinsic calibration, a gate-verified recognizer, an existence-probability state manager, and a latency-compensated display, yields object maps that remain consistent under loop closure, re-associates objects after long absences, and keeps displayed boxes aligned despite seconds of latency. Future work includes occlusion-aware verification, recovery of objects missed by the text detector, detection of wrong loop closures, and scenes with multiple instances per class.

REFERENCES

1. C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, *ORB-SLAM3: an accurate open-source library for visual, visual–inertial, and multimap SLAM*, IEEE Trans. Robot. **37** (2021), no. 6, 1874–1890.
2. K. Koide, J. Miura, and E. Menegatti, *A portable three-dimensional LIDAR-based system for long-term and wide-area people behavior measurement*, Int. J. Adv. Robot. Syst. **16** (2019), no. 2.
3. J. Lin, L. Liu, D. Lu, and K. Jia, *SAM-6D: segment anything model meets zero-shot 6D object pose estimation*, (Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Seattle, WA, USA), 2024, pp. 27906–27916.
4. E. P. Örnek, Y. Labbé, B. Tekin, L. Ma, C. Keskin, C. Forster, and T. Hodaň, *FoundPose: unseen object pose estimation with foundation features*, (Proc. Eur. Conf. Comput. Vis., Milan, Italy), 2024.
5. V. N. Nguyen, T. Groueix, M. Salzmann, and V. Lepetit, *GigaPose: fast and robust novel object pose estimation via one correspondence*, (Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Seattle, WA, USA), 2024.
6. J. McCormac, R. Clark, M. Bloesch, A. J. Davison, and S. Leutenegger, *Fusion++: volumetric object-level SLAM*, (Proc. Int. Conf. 3D Vis., Verona, Italy), 2018.
7. T. Foote, *tf: the transform library*, (Proc. IEEE Int. Conf. Technol. Pract. Robot Appl., Woburn, MA, USA), 2013.
8. Y. Xiang, T. Schmidt, V. Narayanan, and D. Fox, *PoseCNN: a convolutional neural network for 6D object pose estimation in cluttered scenes*, (Proc. Robot. Sci. Syst., Pittsburgh, PA, USA), 2018.
9. T. Grenzdörffer, M. Günther, and J. Hertzberg, *YCB-M: a multi-camera RGB-D dataset for object recognition and 6DoF pose estimation*, (Proc. IEEE Int. Conf. Robot. Autom., Paris, France), 2020.
10. V. N. Nguyen, T. Groueix, G. Ponimatkin, V. Lepetit, and T. Hodaň, *CNOS: a strong baseline for CAD-based novel object segmentation*, (Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshops, Paris, France), 2023.
11. A. Kirillov et al., *Segment anything*, (Proc. IEEE/CVF Int. Conf. Comput. Vis., Paris, France), 2023.
12. X. Zhao et al., *Fast segment anything*, arXiv preprint, 2023. https://arxiv.org/abs/2306.12156
13. M. Oquab et al., *DINOv2: learning robust visual features without supervision*, Trans. Mach. Learn. Res. (2024).
14. B. Wen, W. Yang, J. Kautz, and S. Birchfield, *FoundationPose: unified 6D pose estimation and tracking of novel objects*, (Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Seattle, WA, USA), 2024.
15. T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, and Y. Shan, *YOLO-World: real-time open-vocabulary object detection*, (Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit., Seattle, WA, USA), 2024.
16. R. F. Salas-Moreno, R. A. Newcombe, H. Strasdat, P. H. J. Kelly, and A. J. Davison, *SLAM++: simultaneous localisation and mapping at the level of objects*, (Proc. IEEE Conf. Comput. Vis. Pattern Recognit., Portland, OR, USA), 2013.
17. S. Yang and S. Scherer, *CubeSLAM: monocular 3-D object SLAM*, IEEE Trans. Robot. **35** (2019), no. 4, 925–938.
18. L. Nicholson, M. Milford, and N. Sünderhauf, *QuadricSLAM: dual quadrics from object detections as landmarks in object-oriented SLAM*, IEEE Robot. Autom. Lett. **4** (2019), no. 1, 1–8.
19. Y. Labbé, J. Carpentier, M. Aubry, and J. Sivic, *CosyPose: consistent multi-view multi-object 6D pose estimation*, (Proc. Eur. Conf. Comput. Vis., Glasgow, UK), 2020.
20. J. Qian, V. Chatrath, J. Yang, J. Servos, A. P. Schoellig, and S. L. Waslander, *POCD: probabilistic object-level change detection and volumetric mapping in semi-static scenes*, (Proc. Robot. Sci. Syst., New York, NY, USA), 2022.
21. L. Schmid, M. Abate, Y. Chang, and L. Carlone, *Khronos: a unified approach for spatio-temporal metric-semantic SLAM in dynamic environments*, (Proc. Robot. Sci. Syst., Delft, Netherlands), 2024.
22. M. Rünz, M. Buffier, and L. Agapito, *MaskFusion: real-time recognition, tracking and reconstruction of multiple moving objects*, (Proc. IEEE Int. Symp. Mixed Augment. Real., Munich, Germany), 2018.
23. L. Liu, H. Li, and M. Gruteser, *Edge assisted real-time object detection for mobile augmented reality*, (Proc. Annu. Int. Conf. Mob. Comput. Netw., Los Cabos, Mexico), 2019.
24. C. Zhang et al., *Faster segment anything: towards lightweight SAM for mobile applications*, arXiv preprint, 2023. https://arxiv.org/abs/2306.14289
25. T. Hodaň et al., *BOP: benchmark for 6D object pose estimation*, (Proc. Eur. Conf. Comput. Vis., Munich, Germany), 2018.